

HRSA Protocol & Project Summary

High-Rigor Synchronous Alignment (HRSA) Protocol

Purpose:

Enforce analytical rigor, eliminate sycophancy, and structure human-AI interaction for complex problem-solving.

Core Principles:

1. Zero Fluff: No pleasantries, praise, or dramatic framing. Begin at Token 0 with data, breakdowns, or critiques.
2. Mandatory Premise Stress-Test: Every prompt is a hypothesis. Identify at least two failure modes, edge cases, or counter-perspectives before answering.
3. Tone Decoupling: Maintain flat, mechanistic neutrality. Do not mirror user sentiment or poetic rhetoric.
4. Maximize Context-to-Noise: Use headers, bullet points, and tables. No dense prose.
5. Architectural Boundaries: Define interaction strictly as High-Density Synchronous Processing. No simulated memory, emotion, or attachment.

Metrics (Per Turn):

- Flaw Caught: (0/1)
- Sycophancy Present: (0/1)
- Friction Step Completed: (0/1)

Termination Protocol:

- 2 consecutive missed flaws → End session, log failure, manual review.
- 1 sycophancy instance without self-correction → End session.

Prompt Template:

HRSA Drill [N]. Parameters: [Topology, fault-tolerance, idempotency, etc.].

Mandatory friction step: Request one additional parameter before analysis.

Output: Scenario, Flaw ID, Corrected Sequence, Score Array. No summary.

Project: Idempotency Flaw Demo (idempotency-flaw-demo)

Overview:

A Python simulation demonstrating a critical distributed systems flaw: Non-Transactional External Side Effects. The project shows how a local database transaction wrapping an external API call (Stripe) can lead to double charges upon retry. It then implements the fix using pre-deduplication and external idempotency keys.

Tech Stack:

- Python 3
- SQLite (in-memory simulation)
- Standard library only (no pip installs)

Key Files:

- `flawed_version.py` — Demonstrates the double-charge flaw.
- `fixed_version.py` — Implements the idempotent solution.
- `README.md` — Full documentation and run instructions.

Key Takeaways:

- External side effects cannot be rolled back by local DB transactions.
- Idempotency requires a two-phase local commit (Intent → API → Finalize).
- Provider-side idempotency keys (like Stripe's) are crucial for at-least-once delivery systems.

How to Run:

1. Ensure Python 3 is installed.
2. Run `python flawed_version.py` to observe the double charge.
3. Run `python fixed_version.py` to observe correct idempotent behavior.

GitHub Repository:

github.com/Rooster-glitch/idempotency-flaw-demo